

Software Test Plan

TUNE-TERPRETER APPLICATION | BACKEND UNIT & INTEGRATION TESTS

VERSION 1.0 | AUTHOR: DANIEL PEREZ | JUNE 2026

1. Objective

Tune-Terpreter is a web application that allows users to search for a song and receive an AI-generated interpretation of that song's lyrics using OpenAI's GPT-4o-mini LLM. The purpose of this test plan is to verify that the application's core features function as intended, handle errors gracefully, and do not produce any unexpected behavior.

2. Scope

In Scope

- `fetch_lyrics(song_title, artist_name)` function – sends song title & artist name to the Genius API and returns the song lyrics
- `interpret_lyrics(lyrics)` function – sends lyrics to OpenAI's LLM and returns an AI-generated interpretation and analysis
- GET / route – renders the home page
- POST /search route – accepts song title and artist name, returns lyrics and an AI-generated interpretation
- Error handling for functions and routes

Out of Scope

- Frontend UI behavior – to be covered in a separate frontend test plan
- Genius API – mocked in all tests to ensure the test suite is fast, reliable, and independent of API usage
- OpenAI API – mocked in all tests to ensure the test suite is fast, reliable, and independent of API usage
- End-to-end testing – to be covered in a separate E2E test plan
- Performance and load testing – out of scope for this test plan as it would require dedicated load testing tools
- Deployment configuration – verified separately via post-deployment smoke testing

3. Testing Strategy

Tune-Terpreter uses an automated testing strategy. The testing suite consists of 22 tests across unit and integration levels.

Unit Testing

Individual functions are tested in isolation with all external dependencies mocked. Verifies that each function produces the correct output for a given input

Integration Testing

Flask routes are tested using the Flask test clients. Verifies that requests are handled correctly end-to-end through the route layer, including request parsing, function calls, and response formatting

Smoke Testing

A subset of critical tests function as smoke tests to verify that the application boots and core functionality work (TC-001 test_home_route & TC-002 test_search_success). If either fails, the build is considered functionally broken and further code changes must be halted until resolved

Regression Testing

The fully automated suite is run before any major change to serve as a regression check

4. Test Cases

TC-001	test_home_route
Module	GET /
Level	Integration
Priority	High
Description	Verifies the home page loads successfully
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Send GET request to /
Expected Results	HTTP 200 response
Status	Pass

TC-002	test_search_success
Module	POST /search
Level	Integration
Priority	High

Description	Verifies that the /search route returns HTTP 200 response with lyrics and an AI-generated response
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Mock fetch_lyrics 2. Mock interpret_lyrics 3. Send POST request to /search with valid song and artist in JSON payload
Expected Results	HTTP 200 response; JSON body contains lyrics and summary keys
Status	Pass

TC-003	test_search_missing_song
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 400 response with an error message when POST request is missing the song title in JSON payload
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Send POST request to /search with only artist name in JSON payload
Expected Results	HTTP 400 response; JSON body contains an error key
Status	Pass

TC-004	test_search_empty_song_string
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 400 response with an error message when POST request contains an empty string as the song title in JSON payload
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Send POST request to /search with an empty string passed as the song title in JSON payload
Expected Results	HTTP 400 response; JSON body contains an error key
Status	Pass

TC-005	test_search_missing_artist
Module	POST /search
Level	Integration
Priority	High

Description	Verifies that the /search route returns HTTP 400 response with an error message when POST request is missing the artist name in JSON payload
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Send POST request to /search with only song title in JSON payload
Expected Results	HTTP 400 response; JSON body contains an error key
Status	Pass

TC-006	test_search_empty_artist_string
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 400 response with an error message when POST request contains an empty string as the artist name in JSON payload
Preconditions	Flask test client; rate limiter disabled via 'client' fixture
Steps	1. Send POST request to /search with an empty string passed as the artist name in JSON payload
Expected Results	HTTP 400 response; JSON body contains an error key
Status	Pass

TC-007	test_search_song_not_found
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 404 with an error message when fetch_lyrics raises a SongNotFoundError
Preconditions	Flask test client; rate limiter disabled via 'client' fixture; fetch_lyrics mocked to raise SongNotFoundError
Steps	1. Mock fetch_lyrics to raise SongNotFoundError 2. Send POST request to /search with valid song and artist in JSON payload
Expected Results	HTTP 404 response; JSON body contains an error key
Status	Pass

TC-008	test_search_lyrics_not_found
Module	POST /search
Level	Integration
Priority	High

Description	Verifies that the /search route returns HTTP 404 with an error message when fetch_lyrics raises a LyricsNotFoundError
Preconditions	Flask test client; rate limiter disabled via 'client' fixture; fetch_lyrics mocked to raise LyricsNotFoundError
Steps	1. Mock fetch_lyrics to raise LyricsNotFoundError 2. Send POST request to /search with valid song and artist in JSON payload
Expected Results	HTTP 404 response; JSON body contains an error key
Status	Pass

TC-009	test_search_genius_service_error
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 502 response with an error message when fetch_lyrics raises a GeniusServiceError
Preconditions	Flask test client; rate limiter disabled via 'client' fixture; fetch_lyrics mocked to raise GeniusServiceError
Steps	1. Mock fetch_lyrics to raise GeniusServiceError 2. Send POST request to /search with valid song and artist in JSON payload
Expected Results	HTTP 502 response; JSON body contains an error key
Status	Pass

TC-010	test_search_openai_service_error
Module	POST /search
Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 502 response with an error message when interpret_lyrics raises OpenAIServiceError
Preconditions	Flask test client; rate limiter disabled via 'client' fixture; fetch_lyrics mocked; interpret_lyrics mocked to raise OpenAIServiceError
Steps	1. Mock fetch_lyrics 2. Mock interpret_lyrics to raise OpenAIServiceError 3. Send POST request to /search with valid song and artist in JSON payload
Expected Results	HTTP 502 response; JSON body contains an error key
Status	Pass

TC-011	test_search_limiting_error
Module	POST /search

Level	Integration
Priority	High
Description	Verifies that the /search route returns HTTP 429 response with an error message when the rate limit is exceeded
Preconditions	Flask test client; rate limiter enabled via 'client_limiting_enabled' fixture
Steps	1. Mock fetch_lyrics 2. Mock interpret_lyrics 3. Send 6 sequential POST requests to /search with valid song and artist in JSON payload
Expected Results	HTTP 429 response; JSON body contains an error key
Status	Pass

TC-012	test_fetch_lyrics_success
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics returns song lyrics with a valid song title and artist name
Preconditions	Mock genius.search_song to return a mock song object with valid lyrics
Steps	1. Mock genius.search_song to return a mock song object with valid lyrics 2. Call fetch_lyrics("Test Song", "Test Artist")
Expected Results	Function returns the mocked lyrics string
Status	Pass

TC-013	test_fetch_lyrics_empty_song_string
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics raises a SongNotFoundError when called with an empty string as the song title
Preconditions	None
Steps	1. Call fetch_lyrics("", "Test Artist") with an empty string as the song title and valid artist name 2. Capture log output
Expected Results	SongNotFoundError is raised; log message contains "Empty string passed as song title" identifier
Status	Pass

TC-014	test_fetch_lyrics_empty_artist_string
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics raises a SongNotFoundError when called with an empty string as the artist name
Preconditions	None
Steps	1. Call fetch_lyrics("Test Song", "") with valid song title and empty artist string 2. Capture log output
Expected Results	SongNotFoundError is raised; log message contains "Empty string passed as artist name" identifier
Status	Pass

TC-015	test_fetch_lyrics_unavailable_lyrics
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics raises a LyricsNotFoundError when Genius does not contain lyrics for valid song request
Preconditions	Mock genius.search_song to return a song object with no lyrics
Steps	1. Mock song object with no lyrics 2. Mock genius.search_song with mock song object 3. Call fetch_lyrics("Test Song", "Test Artist") 4. Capture log output
Expected Results	LyricsNotFoundError is raised; log message contains "lyrics unavailable" identifier
Status	Pass

TC-016	test_fetch_lyrics_song_not_found
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics raises a SongNotFoundError when Genius does not contain the requested song
Preconditions	Mock genius.search_song to return None
Steps	1. Mock genius.search_song with return value set to 'None' 2. Call fetch_lyrics("Test Song", "Test Artist") 3. Capture log output
Expected Results	SongNotFoundError is raised; log message contains "not found on Genius" identifier

Status	Pass
---------------	------

TC-017	test_fetch_lyrics_generic_error
Module	fetch_lyrics function
Level	Unit
Priority	High
Description	Verifies that fetch_lyrics raises a GeniusServiceError when Genius raises a generic exception
Preconditions	Mock genius.search_song to raise a generic Exception
Steps	1. Mock genius.search_song to create an 'Exception' side effect 2. Call fetch_lyrics("Test Song", "Test Artist") 3. Capture log output
Expected Results	GeniusServiceError is raised; log message contains "An unexpected error occurred" identifier
Status	Pass

TC-018	test_interpret_lyrics_success
Module	interpret_lyrics function
Level	Unit
Priority	High
Description	Verifies that interpret_lyrics returns an AI-generated interpretation and analysis of song lyrics
Preconditions	Mock openai.OpenAI client; configure mock to return a valid chat completion response
Steps	1. Mock OpenAI response structure 2. Mock the OpenAI client chat completion 3. Call interpret_lyrics("Test lyrics")
Expected Results	Function returns the mocked AI-generated response string
Status	Pass

TC-019	test_interpret_lyrics_rate_limit_error
Module	interpret_lyrics function
Level	Unit
Priority	High
Description	Verifies that interpret_lyrics raises OpenAIServiceError when OpenAI raises a RateLimitError
Preconditions	Mock openai.OpenAI client; configure mock to raise openai.RateLimitError when chat.completions.create is called
Steps	1. Mock openai.OpenAI and configure it to raise RateLimitError 2. Call interpret_lyrics("Test lyrics")

	3. Capture log output
Expected Results	OpenAIServiceError is raised; log message contains “OpenAI rate limit exceeded” identifier
Status	Pass

TC-020	test_interpret_lyrics_auth_error
Module	interpret_lyrics function
Level	Unit
Priority	High
Description	Verifies that interpret_lyrics raises OpenAIServiceError when OpenAI raises an AuthenticationError
Preconditions	Mock openai.OpenAI client; configure mock to raise AuthenticationError when chat.completions.create is called
Steps	1. Mock openai.OpenAI and configure it to raise AuthenticationError 2. Call interpret_lyrics(“Test lyrics”) 3. Capture log output
Expected Results	OpenAIServiceError is raised; log message contains “OpenAI authentication failed” identifier
Status	Pass

TC-021	test_interpret_lyrics_timeout_error
Module	interpret_lyrics function
Level	Unit
Priority	High
Description	Verifies that interpret_lyrics raises OpenAIServiceError when OpenAI raises an APITimeoutError
Preconditions	Mock openai.OpenAI client; configure mock to raise APITimeoutError when chat.completions.create is called
Steps	1. Mock openai.OpenAI and configure it to raise APITimeoutError 2. Call interpret_lyrics(“Test lyrics”) 3. Capture log output
Expected Results	OpenAIServiceError is raised; log message contains “OpenAI request timed out” identifier
Status	Pass

TC-022	test_interpret_lyrics_generic_error
Module	interpret_lyrics function
Level	Unit
Priority	High

Description	Verifies that interpret_lyrics raises OpenAIServiceError when OpenAI raises a generic exception
Preconditions	Mock openai.OpenAI client; configure mock to raise an Exception
Steps	1. Mock openai.OpenAI and configure it to raise Exception 2. Call interpret_lyrics("Test lyrics") 3. Capture log output
Expected Results	OpenAIServiceError is raised; log message contains "An unexpected error occurred" identifier
Status	Pass

5. Test Environment

Component	Details
Language	Python 3.13
Framework	Flask 3.1.2
Test Runner	pytest 8.4.2
Mocking	pytest-mock 3.15.1
Coverage Tool	coverage 7.11.0 + pytest-cov 7.0.0
External Dependencies	openai, lyricsgenius. Indirect dependencies are listed in requirements.txt
Environment Variables	OPENAI_API_KEY, GENIUS_ACCESS_TOKEN – required for app to start but are mocked for all testing. OPENAI_LLM_MODEL – OpenAI GPT-4o-mini
Environment Management	python-dotenv. Used to load environment variables from .env during local development
Rate Limiting	Flask-Limiter 4.1.1 (configured with explicit memory:// URI for in-memory storage)

6. Entry & Exit Criteria

Entry Criteria

- All dependencies are installed via requirements.txt
- The application runs locally without errors
- All test files are present and discoverable by pytest
- Environment variables are configured
- No open critical bugs that would block testing

Exit Criteria

- All test cases have been executed
- All tests pass with no failures
- Code coverage is at or above 90%
- No open or high severity bugs

7. Risks & Assumptions

Risks

- Mocks reflect Genius and OpenAI behavior at the time of writing. If either API changes its response structure, error types, or rate limit behavior, mocks may no longer accurately represent real behavior. Periodic integration tests against the live APIs are recommended
- Test coverage does not guarantee the absence of bugs as it only verifies the scenarios that have been explicitly tested
- Edge cases not covered by this test plan may exist. Exploratory testing is recommended to supplement these tests
- Flask-Limiter currently uses in-memory storage, which means rate limit counters are not shared across worker processes. In production with multiple workers, rate limiting behavior may not be enforced consistently

Assumptions

- All external dependencies (Genius & OpenAI) are mocked and do not require a live connection to run the test suite
- The test environment matches the production environment in terms of Python version and dependencies
- Requirements are stable. Any changes to feature requirements may require updates to existing test cases